



P1 2026

ESTRUTURAS DE DADOS E ALGORITMOS

PROVA

BALANÇO TÉCNICO | MÓDULO 13

QUETÕES DE MÚLTIPLA ESCOLHA

- 1 Dois algoritmos de ordenação amplamente estudados são o bubble sort e o merge sort. O primeiro compara e troca elementos adjacentes repetidamente; o segundo divide o vetor ao meio, ordena cada metade recursivamente e combina os resultados usando um vetor auxiliar. Os pseudocódigos a seguir descrevem as duas implementações.

```
bubble_sort(v, n):  
  para i de 0 até n-2:  
    para j de 0 até n-2-i:  
      se v[j] > v[j+1]:  
        troca(v[j], v[j+1])
```

```
merge_sort(v, ini, fim):  
  se ini >= fim: retorne  
  mid = (ini + fim) / 2  
  merge_sort(v, ini, mid)  
  merge_sort(v, mid+1, fim)  
  merge(v, ini, mid, fim)
```

Assinale a alternativa que apresenta corretamente a complexidade de tempo e o uso de memória auxiliar de cada algoritmo.

- A Bubble sort: $\Theta(n \log n)$ de tempo e $O(1)$ de memória.
Merge sort: $\Theta(n \log n)$ de tempo e $O(\log n)$ de memória.
- B Bubble sort: $\Theta(n^2)$ de tempo e $O(\log n)$ de memória.
Merge sort: $\Theta(n \log n)$ de tempo e $O(\log n)$ de memória.
- C Bubble sort: $\Theta(n^2)$ de tempo e $O(1)$ de memória.
Merge sort: $\Theta(n^2)$ de tempo e $O(n)$ de memória.
- D Bubble sort: $\Theta(n \log n)$ de tempo e $O(1)$ de memória.
Merge sort: $\Theta(n \log n)$ de tempo e $O(n)$ de memória.
- E Bubble sort: $\Theta(n^2)$ de tempo e $O(1)$ de memória.
Merge sort: $\Theta(n \log n)$ de tempo e $O(n)$ de memória.

Gabarito: E

- ② Uma pilha é uma estrutura de dados que armazena uma coleção de itens de dados relacionados e que garante o seguinte funcionamento: o último elemento a ser inserido é o primeiro a ser removido. É comum na literatura utilizar os nomes **push** e **pop** para as operações de inserção e remoção de um elemento em uma pilha, respectivamente. O seguinte trecho de código em linguagem C++ define uma estrutura de dados pilha utilizando um vetor de inteiros, bem como algumas funções para sua manipulação.

```
#include <iostream>

struct pilha {
    int elementos[100];
    int topo;
};

pilha* cria_pilha() {
    pilha* p = new pilha;
    p->topo = -1;
    return p;
}

void push(pilha* p, int elemento) {
    if (p->topo >= 99)
        return;
    p->elementos[++p->topo] = elemento;
}

int pop(pilha* p) {
    int a = p->elementos[p->topo];
    p->topo--;
    return a;
}
```

O programa a seguir utiliza uma pilha.

```
int main() {
    pilha * p = cria_pilha();
    push(p, 2);
    push(p, 3);
    push(p, 4);
    pop(p);
    push(p, 2);
    int a = pop(p) + pop(p);
}
```

```
    push(p, a);  
    a += pop(p);  
    std::cout << a;  
    return 0;  
}
```

A esse respeito, avalie as afirmações a seguir.

- I. A complexidade computacional de ambas funções push e pop é $O(1)$.
- II. O valor exibido pelo programa seria o mesmo caso a instrução `a += pop(p);` fosse trocada por `a += a;`
- III. Em relação ao vazamento de memória (memory leak), é opcional chamar a operação `delete p`, pois o vetor usado pela pilha é alocado estaticamente.

É correto o que se afirma em

- ☐ A I, apenas.
- ☐ B II, apenas.
- ☐ C I e II, apenas.
- ☐ D II e III, apenas.
- ☐ E I, II e III.

Gabarito: C

- 3 Em um centro de distribuição, volumes de diferentes lotes precisam ser encaminhados por três etapas de triagem: recebimento, consolidação e expedição. Por limitações do sistema, cada etapa armazena temporariamente os volumes em uma estrutura de dados linear, da qual os itens são retirados um a um e imediatamente inseridos na próxima estrutura.

Considere que, se uma sequência representa uma pilha, o topo é o elemento mais à esquerda; e se uma sequência representa uma fila, a sua frente é o elemento mais à esquerda.

Analise o seguinte cenário: “a sequência inicial ficou armazenada na primeira estrutura de dados na seguinte ordem: (L12,L35,L70,L29,L12,L35,L70,L70). Cada volume foi retirado da primeira estrutura de dados e inserido na segunda estrutura de dados, e a sequência ficou armazenada na seguinte ordem: (L70,L70,L35,L12,L29,L70,L35,L12). Finalmente, cada volume foi retirado da segunda estrutura de dados e inserido na terceira estrutura de dados e a sequência ficou armazenada na seguinte ordem: (L70,L70,L35,L12,L29,L70,L35,L12)”.

Qual a única sequência de estruturas de dados apresentada a seguir pode ter sido usada no cenário descrito acima?

- A Fila - Pilha - Fila.
- B Fila - Fila - Pilha.
- C Fila - Pilha - Pilha.
- D Pilha - Fila - Pilha.
- E Pilha - Pilha - Pilha.

Gabarito: A

- ④ Um centro de distribuição desenvolve um sistema para registrar, em tempo real, os volumes que passam por uma esteira de triagem. Para exibir o painel operacional, o sistema precisa acessar rapidamente o volume armazenado em uma posição i (por exemplo, para paginação e ordenação temporária), além de inserir novos registros no final da sequência à medida que os volumes chegam.

Em algumas rotinas de auditoria, pode ser necessário remover ou inserir registros em posições intermediárias.

A equipe considera três representações para essa sequência:

- (i) array de tamanho fixo,
- (ii) lista encadeada e
- (iii) array dinâmico (vetor redimensionável), que dobra sua capacidade quando fica cheio, realocando o bloco e copiando os elementos.

I. Entre as alternativas avaliadas, arrays (fixos ou dinâmicos) tendem a ser mais adequados quando há necessidade de acesso por índice e boa localidade de memória; já listas encadeadas são úteis quando se deseja flexibilidade de tamanho e muitas inserções/remoções com referência ao nó, mesmo sem acesso direto por índice. Em particular, no array dinâmico com política de dobrar a capacidade, a inserção no final tem custo amortizado $\Theta(1)$, apesar de algumas operações custarem $\Theta(n)$ devido à realocação e cópia.

PORQUE

II. Na política de dobrar a capacidade, a realocação ocorre a cada inserção no final, o que implica custo amortizado $\Theta(\log n)$ para uma sequência de n inserções.

A respeito dessas asserções, assinale a opção correta.

- Ⓐ As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.
- Ⓑ As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.

- ☒ C A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.
- ☐ D A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.
- ☐ E As asserções I e II são proposições falsas.

Gabarito: C

- 5 As notações O , Ω e Θ são utilizadas na análise de algoritmos para descrever, respectivamente, o limite superior, o limite inferior e o limite exato do crescimento do tempo de execução em função do tamanho da entrada.

Avalie as afirmações a seguir.

I. $O(n^2)$ estabelece um limite superior assintótico: o algoritmo cresce no máximo tão rápido quanto n^2 para entradas suficientemente grandes.

II. $\Omega(n^2)$ estabelece um limite superior assintótico: o algoritmo nunca gasta mais do que n^2 operações no melhor caso.

III. As notações O , Ω e Θ ignoram constantes multiplicativas: $100n^2$, n^2 e $0{,}01n^2$ são todas descritas como $\Theta(n^2)$.

É correto apenas o que se afirma em

- A I, apenas.
- B I e III, apenas.
- C II e III, apenas.
- D I e II, apenas.
- E I, II e III.

Gabarito: B

- ⑥ Um sistema embarcado de controle de estoque precisa ordenar um array de inteiros representando quantidades de produtos. Por restrições de memória, a equipe optou pelo selection sort, que realiza a ordenação in-place sem necessidade de espaço auxiliar proporcional a n . O código C++ a seguir apresenta a estrutura do algoritmo com o corpo da função `selection_sort` a ser completado.

```
#include <iostream>

void troca(int v[], int i, int j) {
    int tmp = v[i];
    v[i] = v[j];
    v[j] = tmp;
}

void selection_sort(int v[], int n) {
    for (int i = 0; i < n - 1; i++) {
        // implementação
    }
}
```

Considerando que a função deve ordenar o array em ordem crescente, assinale a alternativa que apresenta a implementação correta do corpo do laço de `selection_sort`.

- Ⓐ `int min_idx = 0;`
 for (int j = i + 1; j < n; j++)
 if (v[j] < v[min_idx])
 min_idx = j;
 troca(v, i, min_idx);
- Ⓑ `int min_idx = i;`
 for (int j = i; j < n; j++)
 if (v[j] < v[min_idx])
 min_idx = j;
 troca(v, i, min_idx);
- Ⓒ `int min_idx = i;`
 for (int j = i + 1; j < n; j++)
 if (v[j] < v[min_idx])
 min_idx = j;
 troca(v, i, i + 1);

- Ⓓ `int min_idx = i;`
 `for (int j = i + 1; j <= n; j++)`
 `if (v[j] < v[min_idx])`
 `min_idx = j;`
 `troca(v, i, min_idx);`
- Ⓔ `int min_idx = i;`
 `for (int j = i + 1; j < n; j++)`
 `if (v[j] < v[min_idx])`
 `min_idx = j;`
 `troca(v, i, min_idx);`

Gabarito: E

- ⑦ A recursão de cauda (*tail recursion*) é uma forma especial de recursão em que a chamada recursiva é a última operação executada pela função, sem nenhum processamento pendente após seu retorno. Compiladores que implementam a otimização TCO (*Tail Call Optimization*) são capazes de transformar esse padrão em um laço iterativo, reutilizando o frame atual da pilha de chamadas. Considere as duas implementações de Fibonacci a seguir em C++.

```
int fib(int n) {  
    if (n <= 1) return n;  
    return fib(n - 1) + fib(n - 2);  
}  
  
int fib_tail(int n, int a = 0, int b = 1) {  
    if (n == 0) return a;  
    return fib_tail(n - 1, b, a + b);  
}
```

Avalie as afirmações a seguir.

I. Na versão 2, a chamada recursiva é a última operação da função e não há processamento pendente após seu retorno; portanto, um compilador com TCO pode reutilizar o frame da pilha, reduzindo o uso de memória de $O(n)$ para $O(1)$.

II. A versão 2 é necessariamente mais rápida do que a versão 1 em qualquer compilador C++, pois a recursão de cauda elimina o overhead de empilhamento por definição, independentemente das flags de otimização utilizadas.

III. A versão 1 já é um exemplo de recursão de cauda, pois a instrução `return` é a última linha da função.

É correto apenas o que se afirma em

- Ⓐ I, apenas.
- Ⓑ I e II, apenas.
- Ⓒ II e III, apenas.
- Ⓓ I e III, apenas.

E I, II e III.

Gabarito: A

- ⑧ Memory leak, ou vazamento de memória, é um problema que ocorre em sistemas computacionais quando uma parte da memória, alocada para uma determinada operação, não é liberada quando se torna desnecessária. Em C++, esse tipo de problema está quase sempre relacionado ao uso incorreto dos operadores `new` e `delete`. Esse erro de programação pode levar a falhas no sistema se a memória for completamente consumida.

A partir dessas informações, assinale a opção que apresenta um trecho com memory leak.

- Ⓐ `void f() {
 char* s = new char[50];
 delete[] s;
}`
- Ⓑ `int f() {
 float* a;
 return 0;
}`
- Ⓒ `int f(const char* data) {
 char* s = new char[50];
 int size = std::strlen(data);
 if (size > 50)
 return -1;
 delete[] s;
 return 0;
}`
- Ⓓ `int* f(int n) {
 int* num = new int[n];
 return num;
}
int main() {
 int* num = f(10);
 delete[] num;
 return 0;
}`
- Ⓔ `void f() {
 char* m = new char[10];
 char* n = new char[10];
}`

```
        delete[] m;  
        m = n;  
        delete[] m;  
    }
```

Gabarito: C

QUESTÃO DISCURSIVA

- ① Roteadores de rede precisam gerenciar filas de pacotes para controlar o tráfego entre interfaces. Uma estratégia amplamente adotada é o FIFO (*First In, First Out*): pacotes são enfileirados na ordem de chegada e transmitidos nessa mesma ordem. Em situações de congestionamento, o roteador pode precisar descartar pacotes quando a fila atinge sua capacidade máxima.

Um engenheiro implementou uma fila circular com vetor de tamanho MAX em linguagem C++. A estrutura e as operações são definidas abaixo, mas a função `enfileira` está incompleta:

```
const int MAX = 6;

struct Fila {
    int dados[MAX];
    int inicio, fim, tamanho;
};

void inicializa(Fila& f) {
    f.inicio = f.fim = f.tamanho = 0;
}

// Retorna true se enfileirou com sucesso, false se a fila
// estava cheia
bool enfileira(Fila& f, int val) {
    if (f.tamanho == MAX) return false;
    f.dados[f.fim] = val;
    /* LINHA A */
    f.tamanho++;
    return true;
}

int desenfileira(Fila& f) {
    if (f.tamanho == 0) return -1;
    int val = f.dados[f.inicio];
    f.inicio = (f.inicio + 1) % MAX;
    f.tamanho--;
    return val;
}
```

Com base nas informações apresentadas, faça o que se pede nos itens a seguir.

- a) Escreva a instrução que deve substituir o comentário `/* LINHA A */` para que o avanço do ponteiro `fim` respeite o comportamento circular da fila.
- b) Simule a execução da seguinte sequência de operações sobre uma fila inicializada com `MAX = 6`:
- `enfileira(10),`
 - `enfileira(20),`
 - `enfileira(30),`
 - `desenfileira(),`
 - `desenfileira(),`
 - `enfileira(40),`
 - `enfileira(50),`
 - `enfileira(60),`
 - `enfileira(70),`
 - `enfileira(80).`

Ao final, indique os valores de `inicio`, `fim` e `tamanho`, e apresente o conteúdo do vetor `dados`.

- c) Responda, em poucas palavras, quais são as diferenças entre uma implementação de fila simples e fila circular, ambas utilizando arrays. Sua comparação deve abordar **complexidade de operações** e **utilização eficiente de memória**.